Competing Bandits in Matching Markets

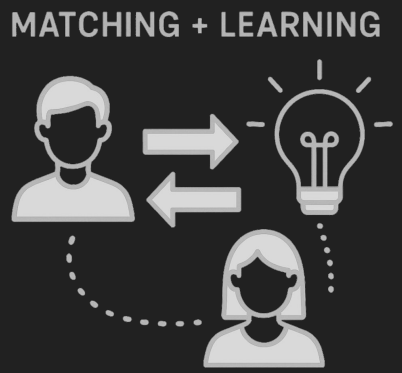
Allizha Theiventhiram, Flaminia Trinca

Final Presentation

Trustworthy AI Systems Seminar, JMCS 2025

Introduction

"Imagine a job market where workers don't know which jobs they like best, but firms already know exactly which workers they prefer. How should such a market operate?"



Agents (workers) do **not** know their true payoffs
→ must **learn** through interaction.

Paper: *Competing Bandits in Matching Markets*
→ explores how **bandit algorithms** interact with **stable matching mechanisms**.

Our contribution:
✓ reproduce paper's results
✓ implement & test **extensions** (new regret, random markets, two-sided learning)

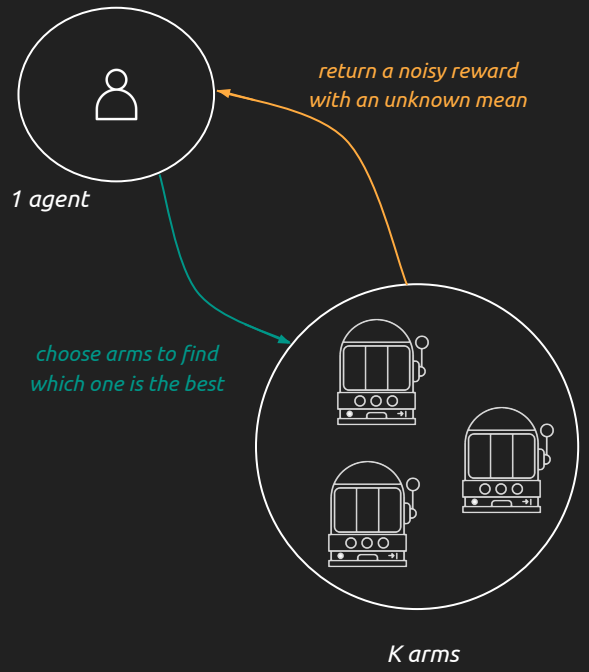
Multi-Armed Bandits

"Learning under uncertainty"

- **One agent** repeatedly chooses among **K arms**.
- Each arm gives a **noisy reward** with an unknown mean
- **Goal**: learn which arms are good by trying them out
- Key challenge:

  Exploration **vs. Exploitation**

  (try new arms vs. stick to the "best-so-far")



Multi-Armed Bandits

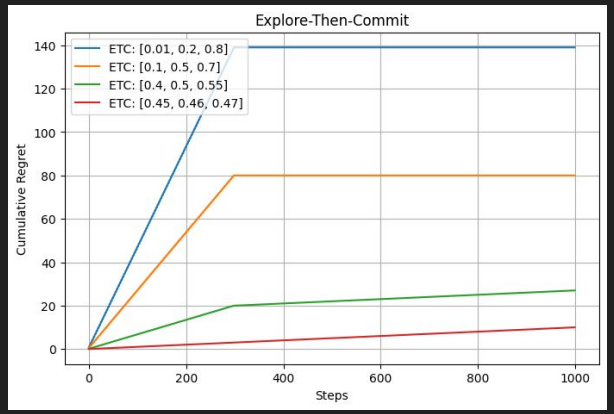
Key Algorithms

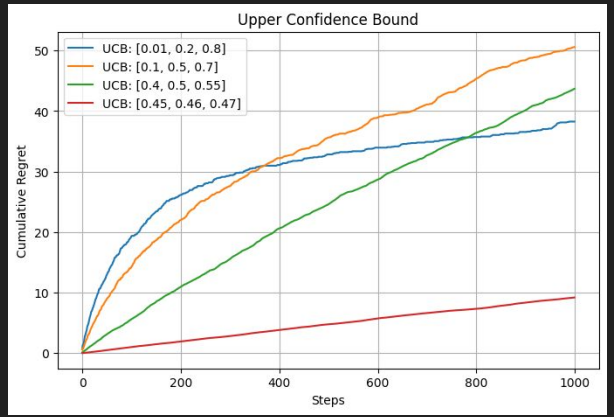
ETC — Explore Then Commit → **"learn once"**

- **Explore:** explore each arm **h** times; try to estimate the expected mean of each arm
- **Commit:** pick the arm with the **best estimate of expected mean,** commit forever
- **Simple but rigid** — one mistake → stuck forever.

UCB — Upper Confidence Bound → **"learn continuously"**

- At each round, compute the **score = mean estimate + exploration bonus**
  - **exploration bonus** depends on the number of times an arm is chosen, and it shrinks over time, which creates this logarithmic regret
    - if an arm is chosen multiple times, uncertainty (=bonus) shrinks
    - if an arm is rarely played, the algo is "optimistic" = plays the arm
- Balances **exploration** (uncertainty term) & **exploitation**.
- **Adaptive and self-correcting**.



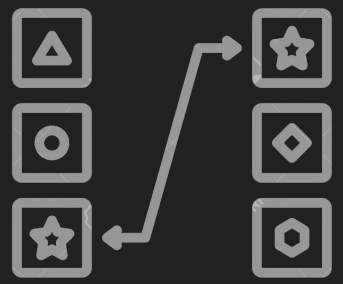


Stable Matching

"Who gets matched with whom"

The goal

Match **agents** (people, students…) to **arms** (jobs, schools…) so that **no two participants prefer each other over their current partners** (== no blocking pairs)



Gale–Shapley Algorithm

Agents have preferences over arms (ex: p1: a1 > a2 > a3)
Arms have preferences over agents (ex: a1: p1 > p2 > p3)

- Agents **propose** to their favorite choice.
- Arms **accept the best proposal so far**.
- Rejected agents try their next option.
- The process stabilizes when **no one wants to deviate**

Output: the **agent-optimal stable matching** (optimal for the proposing side)

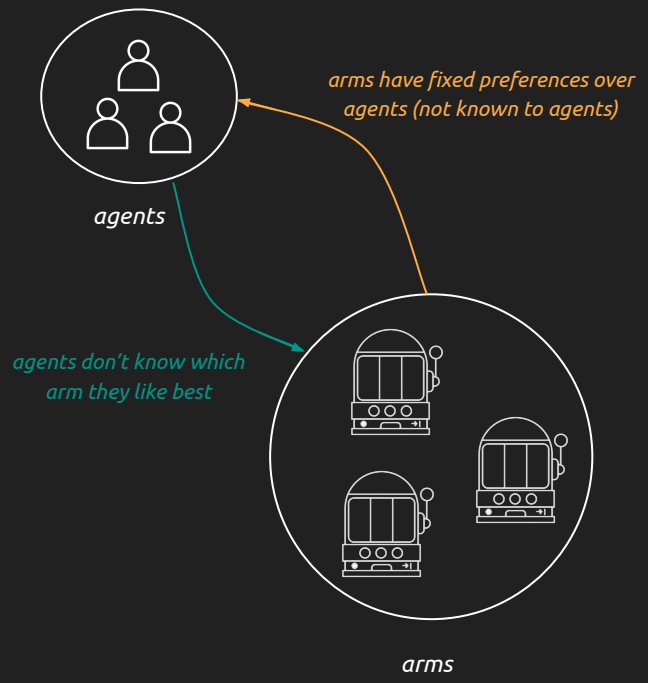
Centralized Platform

"Learning + Matching at Each Round"

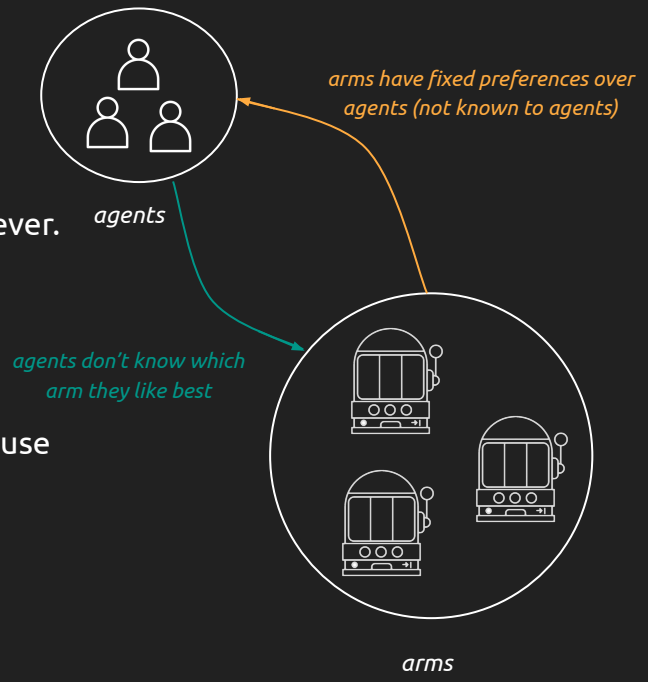
In the **matching-market bandit setting**, each agent pi must **learn their own preference** over arms aj, because the reward Xi,j(t) is unknown to them.

Core idea:
The platform coordinates the entire market:

1. **Multiple Agents learn** their rewards from arms (not known beforehand)
2. **Multiple Arms submit rankings** (based on ETC or UCB)
3. **Platform runs Gale–Shapley** to assign arms to agents
4. **Rewards are observed**, learning continues



Centralized Algorithms



Centralized ETC

- **Exploration**: platform cycles agents over all arms **hK times**
- **Commit**: agents rank arms → one Gale–Shapley run → fixed matching forever.

→ **"explore once, commit forever"**

→ **Problem:** we need to know h in advance for ETC to converge to a plausible result, and h depends on

- n: the horizon (total number of rounds)
- delta: the minimum reward gap (agent's don't know that, because that's exactly what the algo wants to learn)

Centralized UCB

→ **Every round:**

- agents rank arms by **UCB rankings**, which reflect
  what is known (empirical means) and
  what is still uncertain (confidence radius, how many times has an arm been picked or not)
- Platform runs Gale–Shapley: agents provide UCB rankings and fixed arm preferences
- agents receive rewards and update statistics

Regret Notions

"How do we measure whether a matching-bandit algorithm made good or bad choices?"

Agent-optimal stable regret

$$\overline{R}_i(t) = t, \mu_i(\overline{m}(i)) - \sum_{s=1}^{t} \underbrace{X_{i,m_s(i)}(s)}_{reward}.$$

*the **optimal regret** of the proposing side (agents) is the **pessimal regret** of the receiving side (arms)*

"the **best** stable partner the agent could possibly get in *any* stable matching"

Agent-pessimal stable regret

$$\underline{R}_i(t) = t, \mu_i\left(\underline{m}(i)\right) - \sum_{s=1}^{t} \underbrace{X_{i,m_s(i)}(s)}_{reward}$$

Every stable matching lies between the agent-optimal and the agent-pessimal stable matching

"the **worst** stable partner the agent could possibly get in *any* stable matching"

Example 2: Dependence on the Reward Gap (Δ)

Setup

2 agents (p1, p2) and 2 arms (a1, a2)

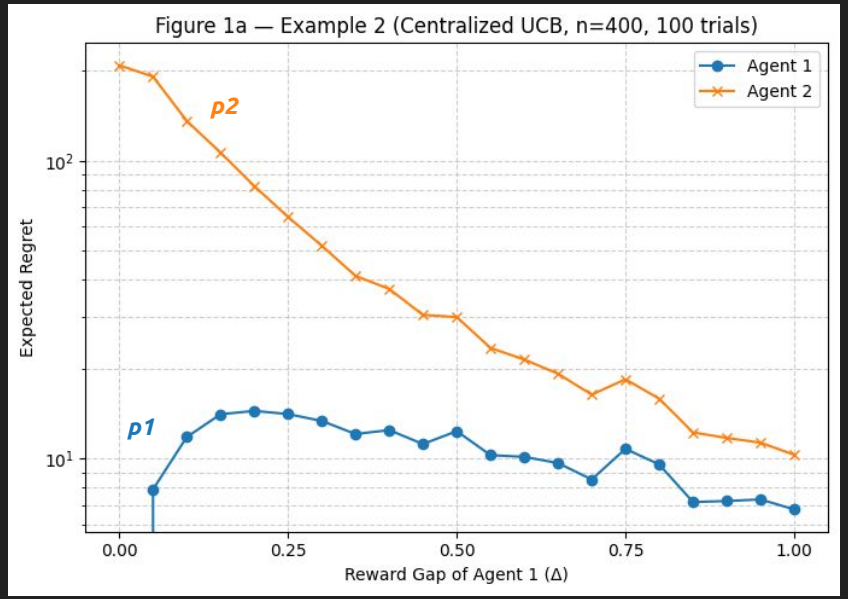
- p1: a1 > a2 → reward for a1 = Δ (for a2=0)
- p2: a2 > a1 → reward for a2 = 1 (for a1=0)

Both arms prefer **p1 over p2**

- Run **Centralized UCB** for n = 400, 100 trials
- Measure **slot-arm regret** (reference arm = same-index arm):
  - *Reference arm for agent i:* ***arm i***
  - *In this example: reference arm =* ***best arm*** *=* ***unique stable partner***

Findings

- **Small Δ → difficult discrimination → higher regret**
- **Large Δ → easier discrimination → lower regret**
- Reproduced curves math the paper's Figure 1a



Δ: *the reward gap for agent 1 between its preferred and non-preferred arms*

Example 6: Linear Agent-Optimal Regret

Setup

- 3 agents (p1, p2, p3) and 3 arms (a1, a2, a3)

Agents' preferences

- p1: a1 > a2 > a3
- p2: a2 > a1 > a3
- p3: a3 > a1 > a2

Arms' preferences

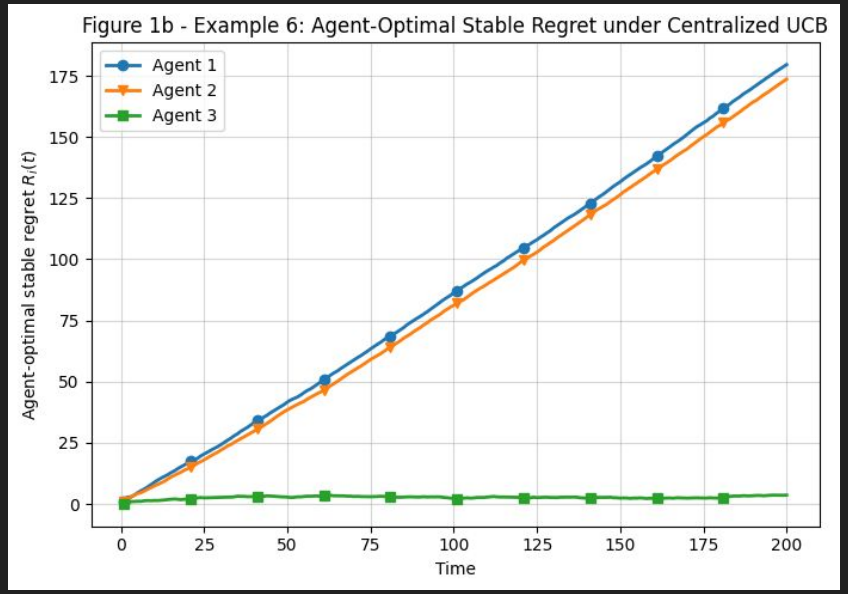
- a1: p2 > p3 > p1
- a2: p1 > p2 > p3
- a3: p3 > p1 > p2

Reward means

- p1: [2, 1, 0]
- p2: [1, 2, 0]
- p3: [1, 0, 1.05]

Experiment

- Centralized UCB
- 200 rounds, 100 trials
- Measure **agent-optimal stable regret**



Findings

everything goes wrong it p3 prefers a1 over a3.

- **p3**: small regret (clear best stable partner)
- **p1** & **p2**: regret grows **linearly**
- Confirms the paper's result:
  Under certain preference structures, **centralized UCB fails to converge** to the agent-optimal stable match.

Example 7: Global Preferences

Setup

20 agents, 20 arms

- All agents rank: **a1 > a2 > … > a20**
- All arms rank: **p1 > p2 > … > p20**
- Unique stable matching: **(p_i, a_i)**

Reward means

- Uniform decrease with gap = 0.1
- μ_i = [2.0, 1.9, 1.8, …, 0.1]

Findings

Top-ranked agents (e.g., p1-p5)

→ **high penalty if they pick a lower ranked arm → large positive regret**

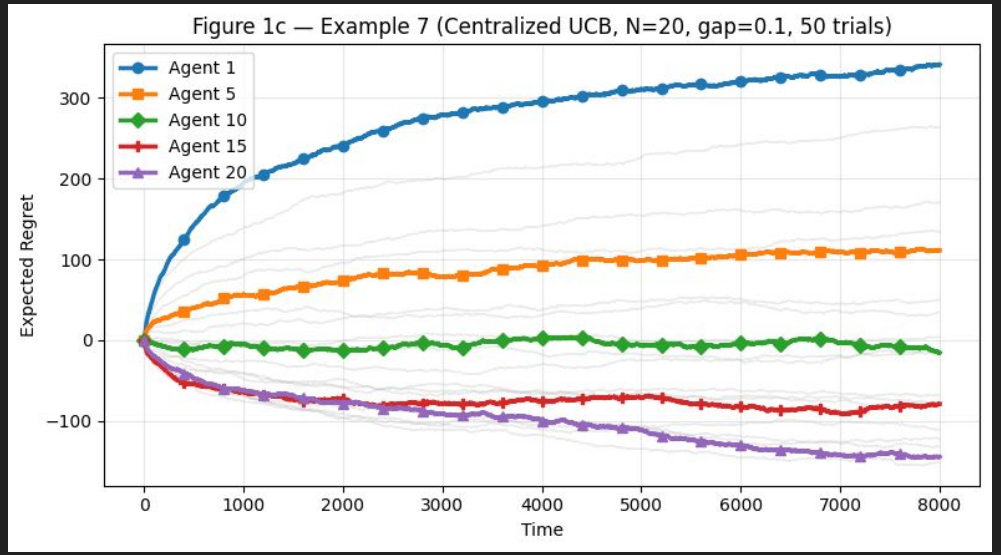
Middle agents (e.g. p6-p14)

→ moderate baseline reward. sometimes better, sometimes worse → **near zero regret**

Low-ranked agents e.g. p15-p20)

→ **assigned arm is bad** → exploration often gives them better arms → **negative regret**

Experiment: Centralized UCB, n = 8000 rounds, 50 trials



The paper's message:

"Market structure crucially affects learning"

"UCB can fail or succeed depending on preferences"

But what happens beyond the paper?

Extension A: Optimal-Matching Regret

"Measure matching accuracy, not reward quality"

$$R_{opt}(T) = \sum_{t=1}^{T} 1\{m_t \neq m^*\}$$

mt = matching at round t, **m*** = true agent-optimal stable matching

Intuition

- Counts rounds where the algorithm outputs the wrong matching
- Ignores reward values → focuses purely on structural correctness

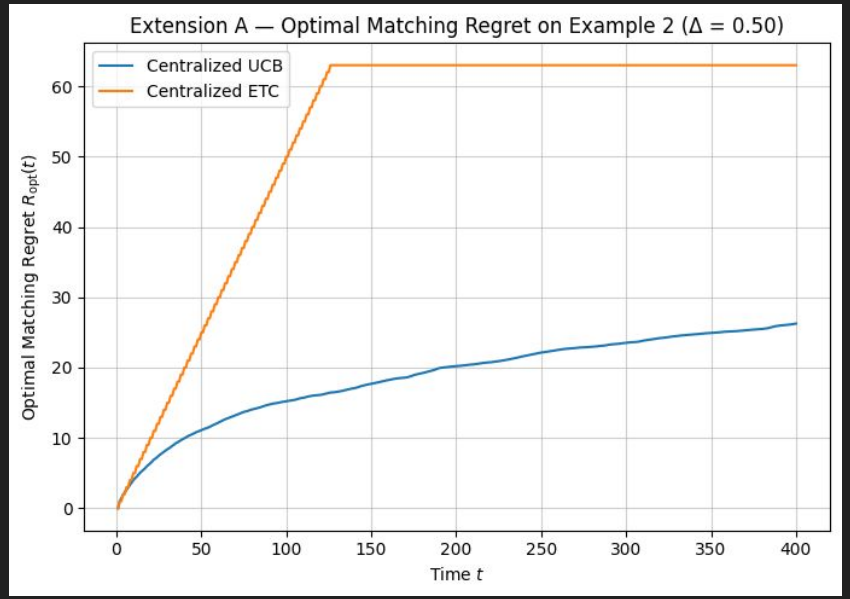
Results on Example 2 (Δ = 0.5) :

Centralized UCB - Ropt(t) increases slowly then stabilizes

- Corrects early mistakes → maintains the right matching
  → **UCB** is **adaptive** and recovers

Centralized ETC - Ropt(t) grows almost linearly

- Early noise during exploration → wrong commitment → persistent mismatch
  → **ETC is brittle** — one early error leads to long-term structural failure



Extension B: Random Two-Sided Markets

"Study algorithm behavior in general, unstructured markets, not hand-crafted examples"

Setup

Generate random two-sided instances with:

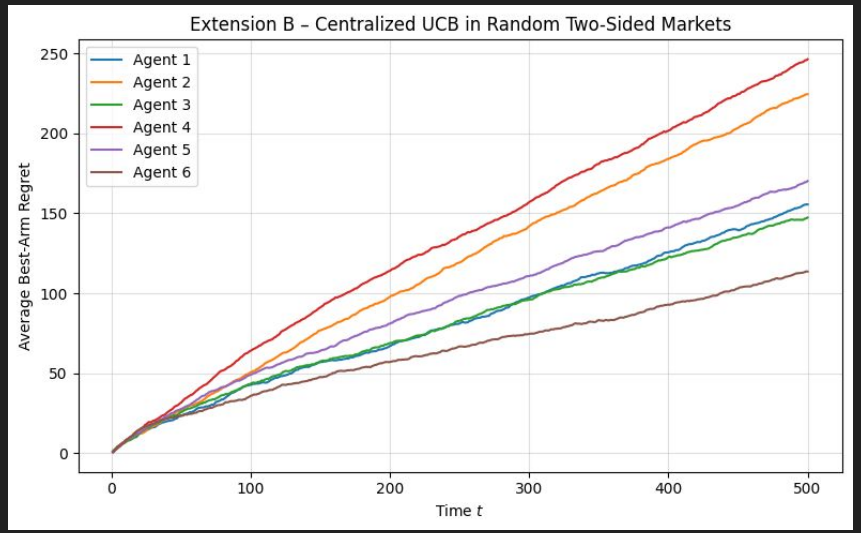
- Random agent and random arm preferences
- Reward means consistent with rankings

Results (UCB: best-arm regret)

- Regret grows **roughly linearly** for most agents
- Large variability: some agents consistently win conflicts, others lose repeatedly
- Behavior driven entirely by random asymmetries in both sides' preferences

Insight

- Realistic markets contain conflict
- Matching constantly reshuffles → unstable learning



Extension B: UCB vs ETC in Random Markets

"Evaluate centralized UCB and ETC in fully random, unstructured two-sided markets"

Setup

Generate random two-sided instances with:

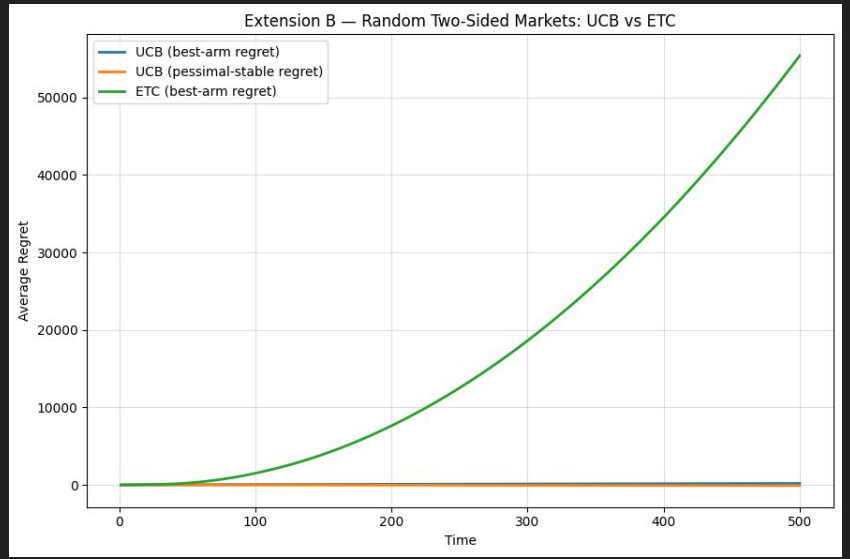
- Random agent and random arm preferences
- Reward means consistent with rankings
- Compute **regret** using **both best-arm** and **pessimal-stable** benchmarks

Finding

- **ETC**: catastrophic (wrong commit → persistent error)
- **UCB**: better but still grows steadily
- Neither algorithm reliably finds stable/optimal outcomes

Insight

- Paper's nice behavior depends heavily on structured markets



Extension C: Two-Sided ETC

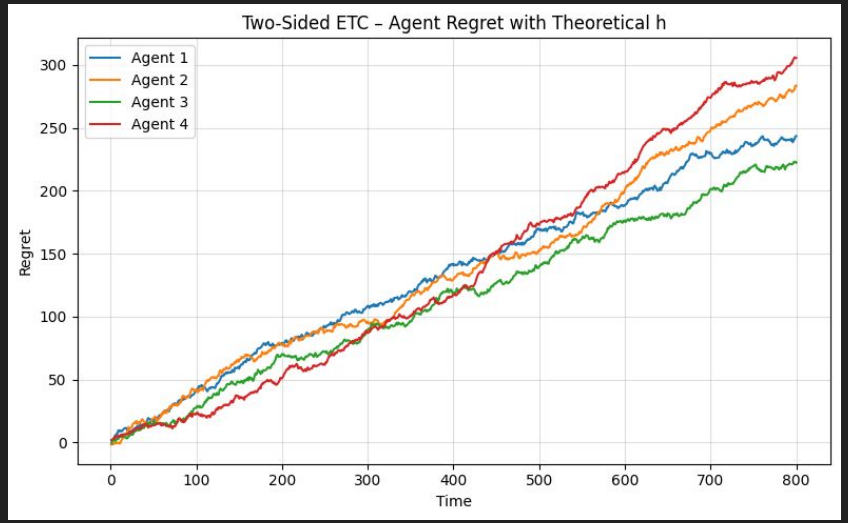
"Study ETC when both sides (agents + arms) learn their preferences"

Theoretical two-sided h (number of rounds) is often larger than the time horizon
→ the algorithm remains entirely in exploration

Setup
4 Agents, **4** Arms
Theoretical h = **3044 when** the entire horizon = **800**

Results
- No commit phase occurs (h > n)
- Agents rotate over arms with no exploitation
- Regret grows almost perfectly linearly for all agents

Takeaway
- Two-sided ETC with theoretical h is not practical — exploration dominates and learning never stabilizes



Conclusion

"Matching + learning is powerful but fragile — naive bandits are not enough."

Key Messages

- Combining **bandit learning** with **stable matching** creates rich dynamics with surprising failure modes.
- **Centralized UCB** works well in structured examples but can fail dramatically (e.g., Example 6: *linear stable regret*).
- **ETC is brittle:** one wrong commit → long-term instability.

Our Contributions

- Reproduced the paper's core results (Examples 2, 6, 7).
- Introduced new evaluations:
  a. optimal-matching regret,
  b. random two-sided markets,
  c. two-sided ETC.

Insights from Extensions

- **Optimal-matching regret** exposes structural errors (UCB recovers, ETC does not).
- **Random markets** generate instability + inequality; algorithms struggle without structure.
- **Two-sided learning** makes ETC essentially unusable (h too large, noisy rankings).

Final Takeaway

To build **trustworthy AI systems** that make real-world matches (jobs, admissions, platforms), we need algorithms that **learn continuously, adapt to noise, and remain stable** - simple bandit-based matching algorithms are **not** enough.